Product 1 — Blinkit (10-Minute Grocery) — System

Link: https://miro.com/app/board/uXjVH8pe6Co=/?share_link_id=782694166030

Product 1 — Blinkit (10-Minute Grocery) — System Design

Step 1: Scope Sheet

Functional requirements (exactly what the brief asks for, nothing more):

1. User opens the app → sees the catalogue for the nearest dark store (location-based)
2. User searches for an item (e.g. "milk") → sees results with price and live stock
3. User adds items to cart
4. User places the order with payment
5. User views live order status: packed → rider assigned → delivered

Non-functional requirements:

Constraint	Value	Reasoning
Scale	~5M DAU across metros	Blinkit-scale quick-commerce assumption
Read vs write	**Read-heavy**, ~20:1	Everyone browses/searches; only a fraction convert to an order. Catalogue and search reads dwarf order writes.
Latency	Catalogue/search < 200ms; order placement < 500ms	10-min delivery promise means the app itself can't add friction — search has to feel instant
Availability	99.9% for browsing, 99.95% for payments	Browsing can tolerate a blip; payment/order path needs stronger durability since money is involved

---

Step 2: API Table

API	Method	Purpose	Internal/External	Payload (brief)
`GET /catalogue?storeId=`	GET	Fetch catalogue for user's nearest dark store	Internal	storeId → list of products with price/category

`POST /location/resolve`	POST	Resolve user's lat/long to nearest dark store	**External** (Google Maps API)	lat, long → storeId, ETA
`GET /search?query=&storeId=`	GET	Search products within a store's inventory	Internal	query, storeId → matched products + stock
`POST /cart`	POST	Add/update item in cart	Internal	userId, productId, qty
`POST /orders`	POST	Place order (creates order, triggers payment)	Internal	userId, storeId, cartItems, addressId
`POST /payments/charge`	POST	Charge the user	**External** (Razorpay/Stripe)	orderId, amount, paymentMethod
`GET /orders/{id}/status`	GET	Poll current order status	Internal	orderId → status enum
`POST /orders/{id}/status-update` (webhook)	POST	Rider app pushes status change	Internal	orderId, newStatus, riderId

A quick gut-check most people skip: notice `GET /catalogue` and `GET /search` are true GETs (no state change), while placing an order and charging payment are POSTs. That distinction is one of the most common places interviewers catch people out.

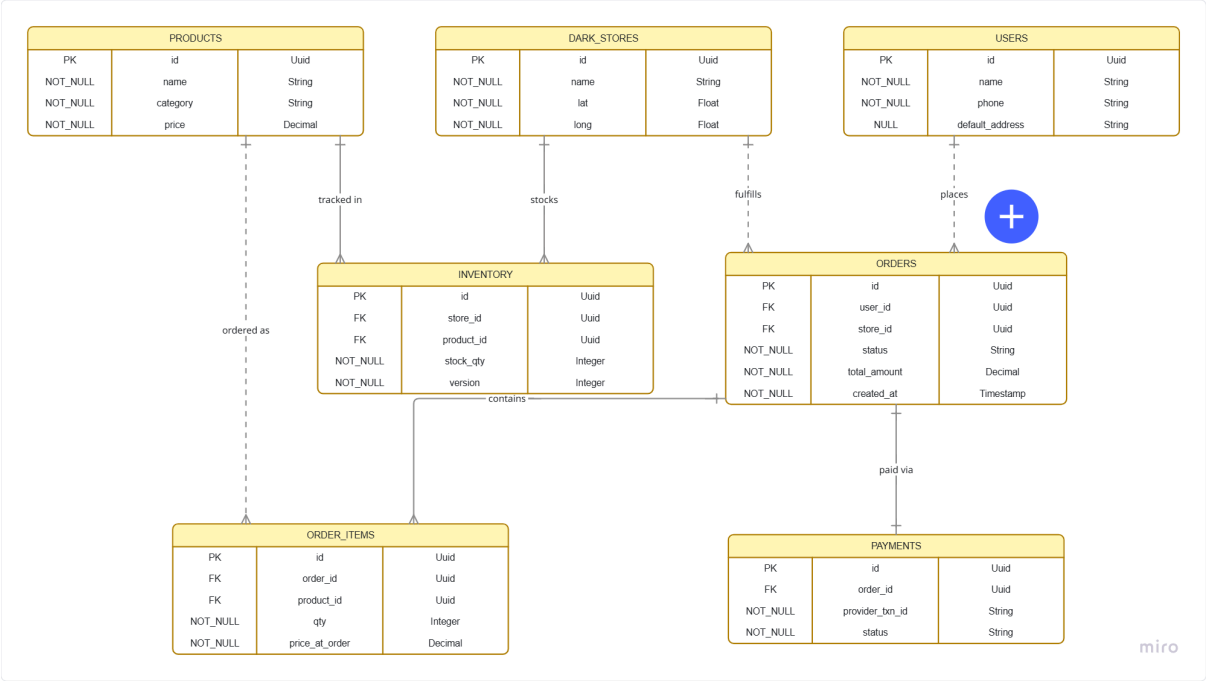
Step 3: Database Design

Reasoning per table:

- **users, dark_stores, products, orders, order_items, payments → SQL (Postgres)**. All predictable, same-shape rows with strict relationships (a user has many orders, an order has many order_items) — exactly the case for foreign keys and joins.
- **inventory** → SQL as well, but it's the most write-contended table in the whole system (see trade-offs below) — every order touches it.

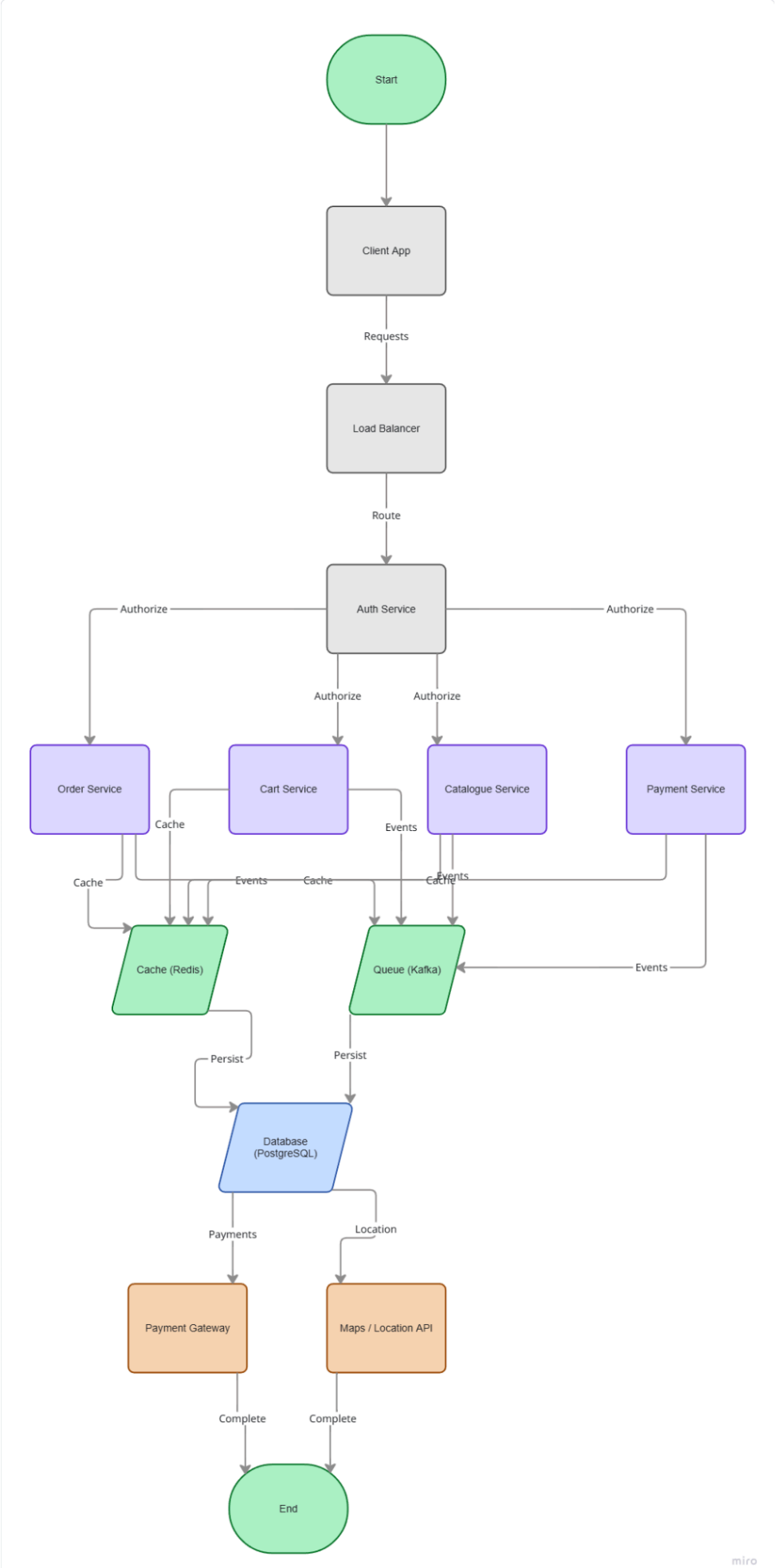
- **cart → NoSQL / Redis key-value**, not a permanent SQL table. It's ephemeral, per-user, changes constantly, and doesn't need joins or durability — a classic "flexible, throwaway data" case for NoSQL over SQL.

Here's the ER diagram for the durable (SQL) side:



The key foreign key: `inventory.store_id` links stock to a specific dark store — that's what makes "what's in stock near me" a fast, indexed lookup rather than a scan. `inventory.version` is there on purpose (see trade-offs below).

Step 4: Architecture Diagram


Start
Client App
Requests
Load Balancer
Route
Auth Service
Authorize
Authorize
Authorize
Authorize
Authorize
Order Service
Cart Service
Catalogue Service
Payment Service
Cache
Events
Cache
Events
Cache
Events
Cache
Cache (Redis)
Queue (Kafka)
Events
Persist
Persist
Database (PostgreSQL)
Payments
Location
Payment Gateway
Maps / Location API
Complete
Complete
End
miro

Cache contents: dark-store catalogue (changes rarely — TTL of minutes) and stock counts (changes often — TTL of seconds, invalidated on every order).

Queue contents: order-placed events (fan out to notification service and rider-assignment), and inventory-decrement events.

Step 5: Trade-offs Note

1. **Race condition — two users grab the last packet of milk simultaneously.** Both requests hit `inventory` at nearly the same instant. Fix: use an atomic decrement with optimistic locking — the `version` column on `inventory` means the second transaction's `UPDATE ... WHERE version = X` fails and retries, sees stock at 0, and returns "out of stock" *before* the payment API is ever called. This is the same pattern as a seat-booking race condition.

2. **Evening peak spike on catalogue/search reads.** 6–9pm traffic can spike 5-10x. Fix: catalogue is cache-first (Redis) so the database only gets a cache-miss trickle; stateless catalogue/search service instances scale horizontally behind the load balancer.

3. **Stale cache shows an item as in-stock when it just sold out.** If the Redis TTL is too long, a user could add an out-of-stock item to cart. Fix: keep stock-count TTL very short (a few seconds) and do a final authoritative check against Postgres at order-confirmation time — the cache is for browsing speed, not for the final source of truth.

Product 2 — Splitwise (Group Expense Splitting)

Product 2 — Splitwise (Group Expense Splitting) — DB-Heavy

Step 1: Scope Sheet

Functional requirements:

1. Create a group (e.g. "Goa Trip") and add friends to it
2. Add an expense: who paid, the amount, and how it splits (equal or custom shares)
3. View balances — within a group and overall ("you owe Rahul ₹1,240")
4. Settle up: record that a payment happened outside the app, and update balances
5. Members get notified when someone adds an expense

Non-functional requirements:

Constraint	Value	Reason
Scale	~500K DAU	Smaller and less transactional than a quick-commerce app
Read vs write	Read-heavy, ~5:1	Users check balances often; they add expenses less often — still read-heavy, but far less skewed than Blinkit's browsing traffic
Latency	Balance view < 300ms; expense add < 500ms	No 10-minute delivery pressure, but a laggy balance screen still feels broken
Availability	99.9%	No payment processing happening inside Splitwise itself, so the stakes (and the target) are a notch lower than Blinkit's payment path

Step 2: API Table

API	Method	Purpose	Internal/External	Payload (brief)
`POST /groups`	POST	Create a group	Internal	name, createdBy
`POST /groups/{id}/members`	POST	Add a friend to a group	Internal	groupId, userId

`POST /expenses`	POST	Add an expense (payer, amount, split)	Internal	groupId, paidBy, amount, splitType, participants
`GET /groups/{id}/balances`	GET	View balances within a group	Internal	groupId → per-user net amounts
`GET /users/{id}/balances`	GET	View overall balance across all groups	Internal	userId → aggregated net amounts
`POST /settlements`	POST	Record a settle-up (payment happened outside the app)	Internal	groupId, fromUser, toUser, amount
`POST /notifications/push`	POST	Notify group members of a new expense	**External** (FCM/CleverTap)	userId, message
`GET /groups/{id}/expenses`	GET	View expense history in a group	Internal	groupId → list of expenses

Step 3: Database Design — where most of the marks live

The core modeling challenge is the **many-to-many** relationship: a user belongs to many groups, and a group has many users. That's `group_members` — a join table, exactly the pattern from the ChartDB many-to-many templates. Same shape again for an expense splitting across many users: `expense_shares` joins `expenses` to `users`.

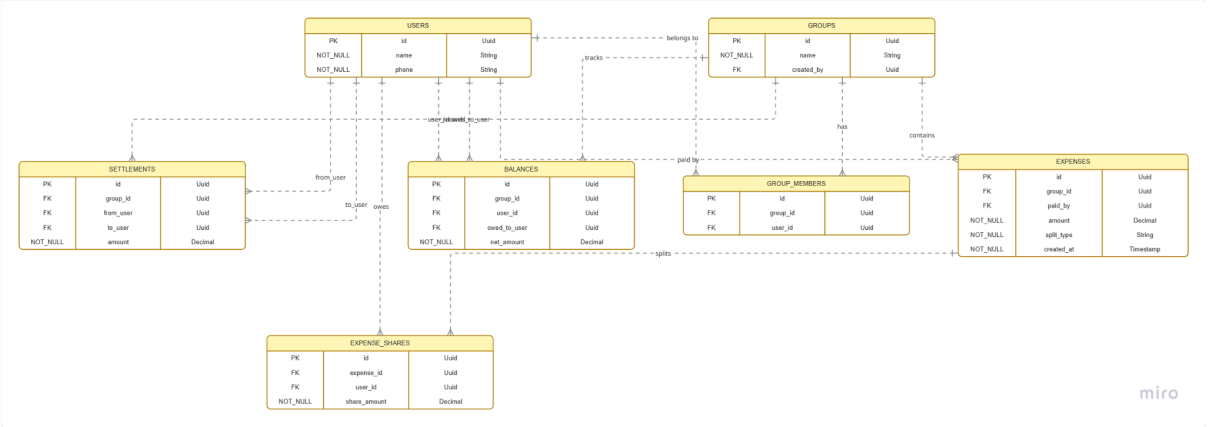
SQL vs NoSQL — everything is SQL (Postgres). Every table here is structured and predictable: users, groups, memberships, expenses, shares, settlements — none of it has the flexible, changing-shape data (like a social feed) that would justify NoSQL. "Everything SQL because the data is predictable" is the reasoned, defensible answer for this product.

The ER diagram above shows the seven tables. A few callouts on the keys:

- `group_members.group_id` + `group_members.user_id` together resolve the group ↔ user many-to-many.

- `expense_shares.expense_id` + `expense_shares.user_id` resolve who owes what for a single expense (an expense split three ways produces three rows here).
- `settlements` and `balances` both carry `from_user`/`owed_to_user` style foreign keys back to `users`, since a debt is always between two specific people, not just "the group."

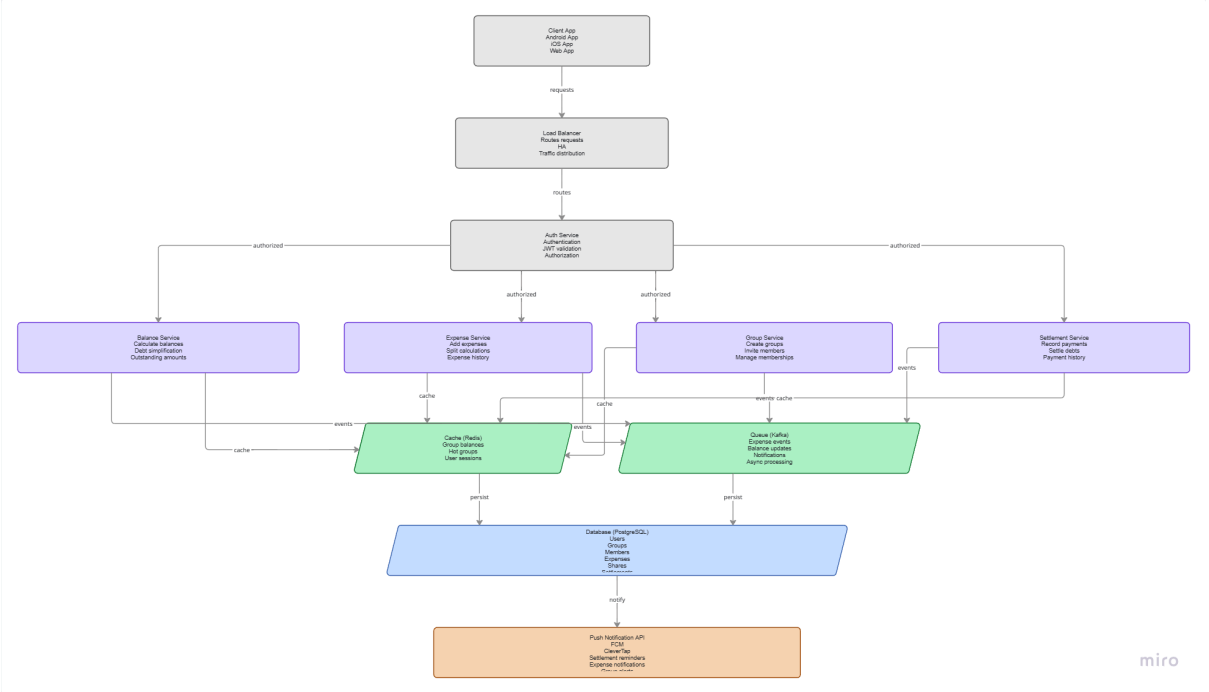
The classic read-vs-write trade-off: do we store balances as a table, or compute them from the full expense history every time someone opens the balances screen?



I chose to **store a materialized `balances` table**, updated transactionally inside the same write that creates an expense or a settlement. Reasoning: balance-view is the single most frequent read in the product (people check "who owes who" far more than they add expenses), and recomputing from every historical expense in a long-running group (a trip that lasted a year, hundreds of expenses) would get progressively slower — exactly the read-heavy profile this system has. The cost is write amplification (every expense write now also updates `balances` rows), which is an acceptable trade given writes are the rarer operation here.

Step 4: Architecture Diagram

Same tiering as Blinkit, but the external dependency is different — there's no payment gateway inside Splitwise, only a **push notification API**.



Cache holds: group balances (short TTL, invalidated on every expense/settlement write — this is the hot read path). **Queue holds:** expense-created events, which fan out asynchronously to the notification service.

The notification flow specifically (Rahul adds an expense, five other members find out): `POST /expenses` writes the expense + shares + updates `balances` in one transaction → an event is pushed to Kafka → the notification service consumes it and calls the external push API once per group member. Doing this via the queue (not synchronously inside the request) means expense creation stays fast even if the push provider is slow or briefly down.

Step 5: Trade-offs Note

1. **Compute vs. store balances** (the read-vs-write trade-off above) — resolved in favor of a stored `balances` table, since balance reads vastly outnumber expense writes.

2. **Double settlement race condition.** Two members could record the same settle-up at nearly the same moment (e.g. both open the app and hit "settle up" for the same debt). Fix: an idempotency key on the settlement request plus a row-level lock on the relevant `balances` entry inside the transaction — the second request sees the already-updated balance and is rejected or becomes a no-op.

3. **Notification fan-out spike in large groups.** A 50-person group adding one expense means 49 notifications firing at once. Fix: this fan-out happens off the request path via the queue, so expense creation itself stays fast regardless of group size or how slow the push provider is at that moment.

Product 3 — Music Streaming (Spotify / JioSaavn)

Product 3 — Music Streaming (Spotify / JioSaavn) — Scale-Heavy

Step 1: Scope Sheet

Functional requirements:

1. Open the app and load a personalised home feed (playlists, recently played, recommendations)
2. Search for a song and play it — playback must start fast
3. Like a song and add it to a playlist
4. The system counts every play (powers charts and artist royalties)

Out of scope (per brief): the recommendation algorithm itself (treated as a service call), offline downloads, podcasts, social features.

Non-functional requirements:

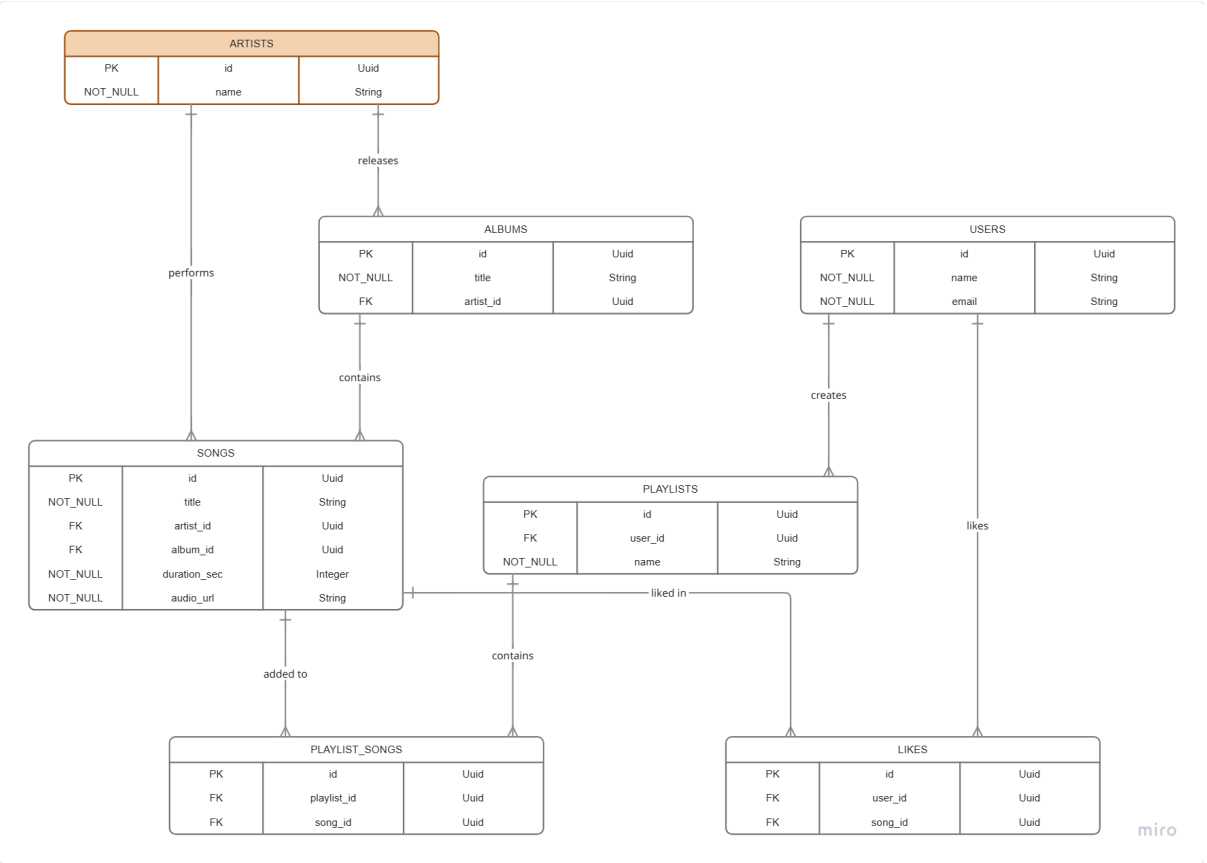
Constraint	Value	Reason
Scale	~100M DAU	Spotify/JioSaavn-scale streaming
Read vs write	**Extremely read-heavy on plays vs. uploads** — roughly 100,000:1	New songs are added rarely; the same catalogue gets replayed by everyone constantly — this is a different read-heavy shape than Blinkit's browsing
Latency	Song must start playing in **< 1 second**; home feed loads in < 300ms	This is the single most important number in the whole design — a slow "play" button is the product failing at its one job
Availability	99.99%	Listening is the core loop; any downtime here is existential for the product, unlike a secondary feature

Being specific here matters — a vague "read-heavy" doesn't tell you *what kind* of read-heavy, and that shapes every later decision (cache-first architecture, CDN for audio, etc.).

Step 2: API Table

API	Method	Purpose	Internal/External	Payload (brief)
`GET /feed/home`	GET	Load personalised home feed	Internal	userId → playlists, recently played, recommendations
`GET /search?query=`	GET	Search songs/artists/albums	Internal	query → matched results
`GET /songs/{id}/stream`	GET	Get the streaming manifest for playback	Internal (points to media CDN)	songId → streaming URL, bitrate options
`POST /songs/{id}/like`	POST	Like a song	Internal	userId, songId
`POST /playlists/{id}/songs`	POST	Add a song to a playlist	Internal	playlistId, songId
`POST /songs/{id}/play-event`	POST	Log a play (fire-and-forget)	Internal	userId, songId, timestamp
`GET /recommendations/{userId}`	GET	Fetch personalised recommendations	Internal (in-house ML service)	userId → ranked song list
`GET /media/{songId}/manifest`	GET	Fetch the actual audio stream	**External** (CDN — CloudFront/Akamai)	songId → audio chunks

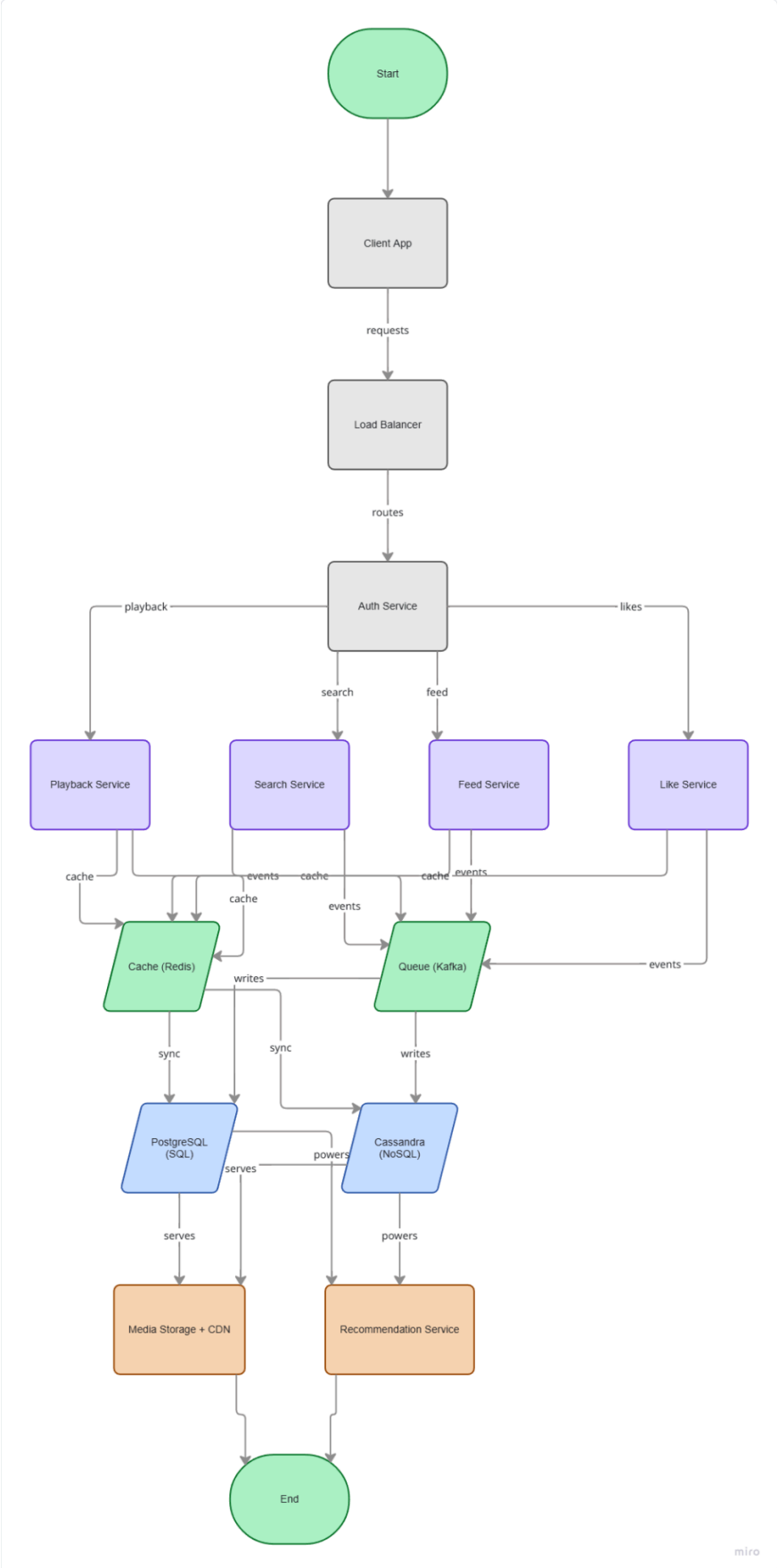
Step 3: Database Design



This is where the "different shapes of data on one screen" problem shows up. The home feed mixes playlists, recently-played, banners, and recommendation cards — genuinely heterogeneous shapes, unlike Splitwise's uniform rows. So the database story here is a **mix**, not one blanket choice:

- **users, artists, albums, songs, playlists, playlist_songs, likes → SQL (Postgres).** These are predictable, same-shape, relational — a song belongs to one artist and one album, a playlist has many songs via a join table (same many-to-many pattern as Splitwise's groups).
- **play_events / play_counts → NoSQL (Cassandra or similar wide-column store).** This is the single highest-write-volume table in the entire system — millions of plays per second at Spotify scale. It doesn't need strong relational consistency (an occasional undercounted play doesn't break anything), so a write-optimized, eventually-consistent NoSQL store fits far better than forcing every play into a transactional SQL row.
- **Audio files are not rows in a table at all.** They live in a separate media storage + CDN layer (same pattern as Airbnb's images on a separate host) — `songs.audio_url` just points to it.`play_events` is deliberately left out of this ER diagram — it lives in Cassandra, not Postgres, and doesn't follow the same relational key structure. It's called out separately below.

Step 4: Architecture Diagram



Cache holds: the assembled home feed per user (short TTL, since it mixes several data sources) and live play/like counters (updated instantly for UI feedback, before they're durably written).

Queue holds: play events and like events — the write-heavy, high-volume side of the system.

Media storage callout: audio files are not database rows — they sit in object storage behind a CDN, exactly like the Airbnb-images-on-a-separate-host pattern from class. `songs.audio_url` just points there.

Step 5: Trade-offs Note

1. **The feed mixes heterogeneous data shapes on one screen.** Playlists, recently-played, and recommendation cards don't share a row structure the way Splitwise's expenses do. Fix: the feed is assembled and cached per-user in Redis rather than joined live across several SQL tables — trading a bit of staleness for a fast, flexible read.

2. **Writing every like and every play straight to the main database would melt it.** At this scale, a naive `UPDATE play_count = play_count + 1` on every play is a guaranteed bottleneck. Fix: reuse the cache-first + queue pattern — increment the counter in Redis instantly (so the UI feels immediate), then push the event to Kafka for asynchronous, batched writes into Cassandra. The user sees an instant update; the durable count catches up moments later.

3. **Recommendation service — build internal or rent external?** I chose **internal (in-house ML service)** for two reasons: (a) user listening history is sensitive, core-IP data that's risky to hand to a third party, and (b) at hundreds of millions of recommendation calls a day, per-call pricing from an external ML API would be cost-prohibitive. The counter-argument for external is faster time-to-ship without needing an in-house ML team — a real trade-off, and either answer is defensible if the reasoning is stated.

Product 4 — Google Calendar

Product 4 — Google Calendar

1. Scope sheet

Functional requirements

1. Create an event (title, time, location)
2. Create a recurring event (e.g. every Tuesday, 5–6 PM)
3. Invite other users to an event
4. Invitee receives a notification and RSVPs (yes/no/maybe)
5. View "my day" — all events for a date, including recurring occurrences and invites
6. Receive a reminder notification 10 minutes before an event starts
7. (Bonus) Register a webhook that fires when a new event is created

Non-functional requirements

- **DAU:** ~5M (mid-size calendar product, not Google-scale)
- **Read vs write:** Read-heavy, roughly 20:1. People check "my day" many times for every one event they create — same reasoning as the BookMyShow browse-vs-book ratio.
- **Latency:** "My day" view < 200ms; event/invite writes < 300ms
- **Availability:** 99.9% — a missed reminder or slow view is annoying, not catastrophic like a payment failure, so this doesn't need payment-grade 99.99%.

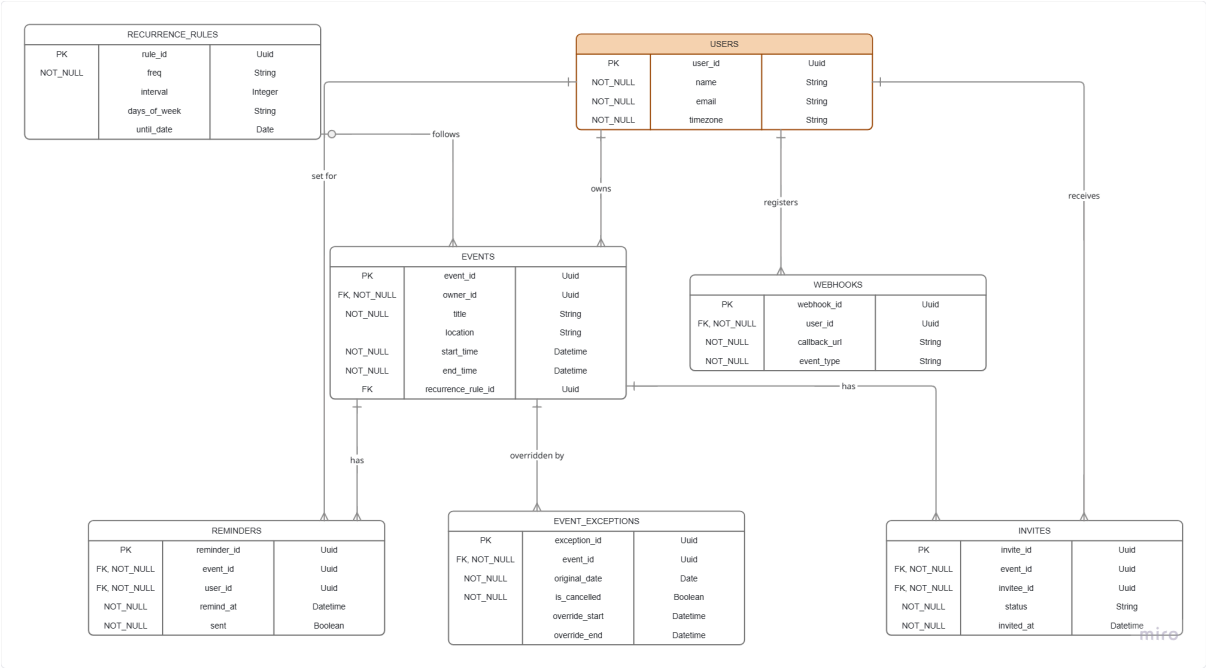
2. API table

API	Method	What it does	Internal / External	Payload (one line)
`POST /events`	POST	Create a single event	Internal	title, start/end time, location, owner_id
`POST /events/recurring`	POST	Create event + recurrence rule	Internal	event fields + freq/interval/until
`PUT /events/{id}`	PUT	Edit an event or one occurrence	Internal	changed fields, optional `occurrence_date` for single-instance edits
`POST /events/{id}/invite`	POST	Invite users to an event	Internal	list of invitee_ids

`PUT /invites/{id}/rsvp`	PUT	Record yes/no/maybe	Internal	status
`GET /users/{id}/day?date=`	GET	Resolved day view (own events + invites)	Internal	—
`POST /webhooks`	POST	Register a callback for event-created	Internal (bonus)	callback_url, event_type
—	—	Push notification delivery	**External** (FCM/APNs)	device token, title, body
—	—	Invite email delivery	**External** (SendGrid)	recipient, template, event details

3. Database design

Design shown in the ER diagram above.

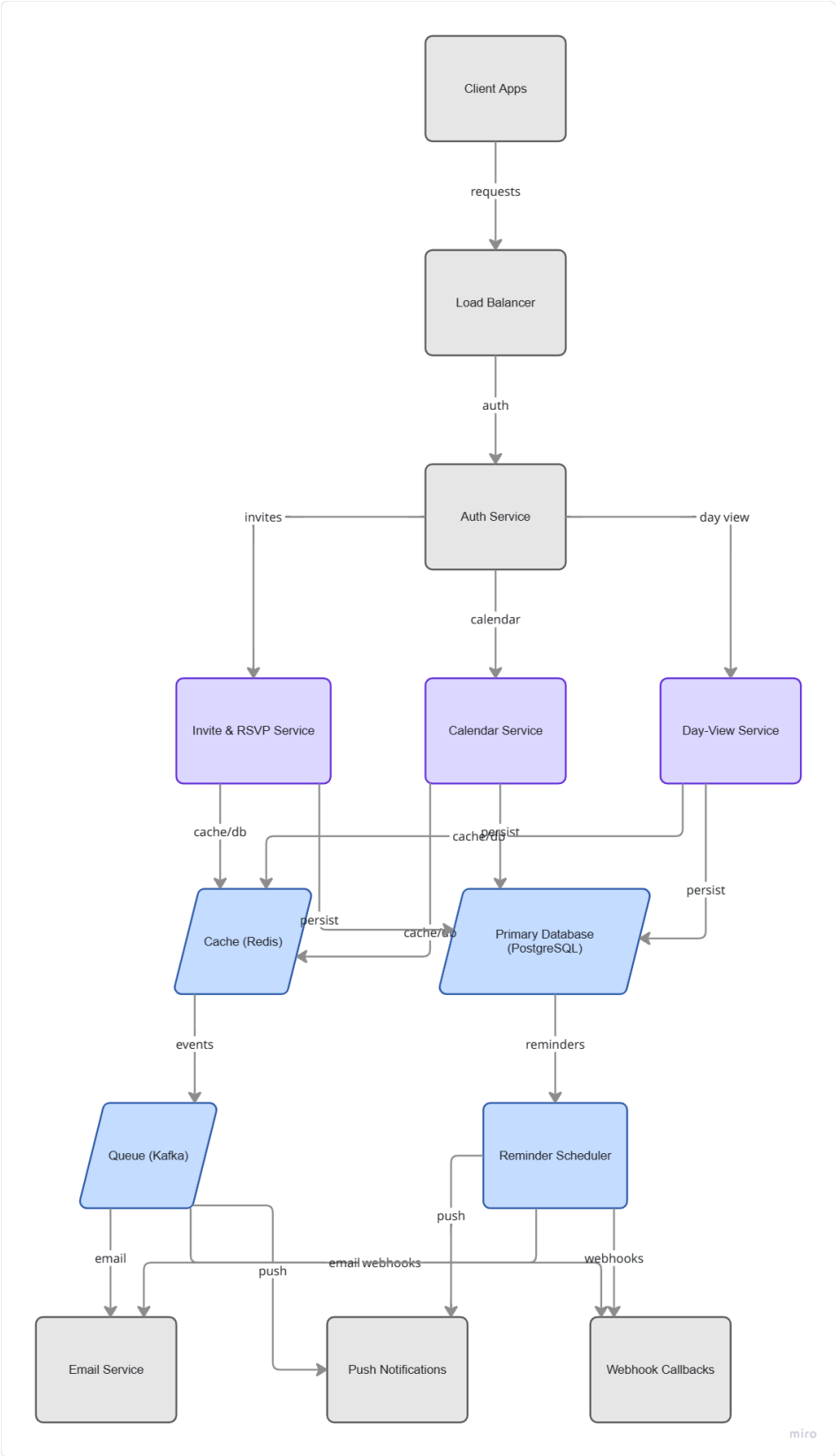


Reasoning:

- **All tables SQL, not NoSQL.** Every relationship here is fixed-shape and needs joins with integrity guarantees (an invite must point to a real event and a real user; a double-booked reminder shouldn't silently duplicate). This is the "everything predictable → SQL" case from the pocket reference, not a feed-shaped problem.

- **Many-to-many via join table:** `INVITES` is the join table between `USERS` and `EVENTS` — same pattern as the ChartDB many-to-many templates, with `status` and `invited_at` as attributes on the relationship itself.
- **Recurring events — one row + a separate rule, not one row per occurrence.** `EVENTS` holds a single row with a foreign key to `RECURRENCE_RULES`. `EVENT_EXCEPTIONS` handles edits or cancellations of individual occurrences by overriding just that date. This keeps storage flat instead of pre-generating 52 rows for a weekly meeting.

4. Architecture



Shown in the diagram above. Key data-flow notes:

- **Cache** holds the *resolved* day view per user per date (after recurrence expansion + exceptions + invites are merged) — the expensive part to compute, not the raw rows.

- **Queue** carries three kinds of async work: invite notifications, reminder dispatch, and webhook deliveries — all decoupled from the request path so a write never waits on a push/email/webhook call.
- **Reminder scheduler** is a separate worker from the queue itself.

5. Trade-offs

1. **Stale day-view cache after an edit.** If someone edits a single occurrence while the day-view cache is warm, the old version could be served briefly. Fix: invalidate the specific `user_id:date` cache key synchronously on every event/exception/invite write that touches that date — not just on a timer.
2. **Reminder scheduling doesn't scale by polling.** Scanning the whole reminders table every minute is fine at 10K events, not at 50M. Fix: keep due-reminders in a time-sorted structure (Redis sorted set) so the scheduler only ever pulls what's actually due right now.
3. **Notification flood on a large invite.** Inviting 500 people to one event shouldn't mean 500 synchronous push/email calls blocking the invite API. Fix: the invite write only touches the `INVITES` table and drops one message per invitee on the queue; notification workers fan out and scale independently, with retries on failure.
4. **Unreliable webhook callbacks.** A third-party callback URL can be slow or down. Fix: deliver via the queue with exponential-backoff retries and a dead-letter queue after N failures — never let a broken external webhook block or slow the core event-creation path.